HW2 — Traditional Software Design vs Function-as-a-Service

System scenario: Hotel Management System | **Language:** C++ (`-O2 -std=c++17`)

Student: Sari Mansour — 322449539

Both architectures implement the **same 10 operations** over the same data model

(`Guest`, `Room`, `Reservation`, `Payment`, `CleaningTask`, `MaintenanceRequest`).

#	Operation	Description
1	add_guest	Register a guest by ID
2	add_room	Add a room (type, price, floor)
3	create_reservation	Reserve an available room for a guest
4	cancel_reservation	Cancel a non-checked-in reservation
5	check_in	Check guest in -> room OCCUPIED
6	check_out	Check guest out -> room CLEANING
7	process_payment	Record a payment for a reservation
8	assign_cleaning_task	Assign staff to clean -> room AVAILABLE
9	report_maintenance	Log maintenance; HIGH/EMERGENCY takes room offline
10	display_available_rooms	Report available room count

Part 1 — Traditional Architecture

A single `HotelSystem` **class owns all data internally** through private `std::map` members; operations are **methods** that directly read and mutate this shared state.

```
class HotelSystem {                    Files:
  private:                               models.h          (data structs)
    map<int,Guest>       guests_;        hotel_system.h    (class declaration)
    map<int,Room>        rooms_;         hotel_system.cpp  (method bodies)
    map<int,Reservation> reservations_;  main.cpp          (demo + benchmark)
    ... payments_, cleaningTasks_,       feature_extension.cpp (Part 3)
        maintenanceRequests_
```

Key property: every lookup uses `std::map::find()` -> **O(log n)**. State is fully encapsulated with no external visibility.

Part 2 — Function-as-a-Service Architecture

The class is replaced by **10 independent, stateless functions**. There are no private members; data lives in an **external** `HotelStorage` struct passed by reference to each function. Each function receives a typed *event*, loads data, validates, performs one task, stores the result, and exits — the FaaS contract.

```
              HotelStorage (external "database")
                        ^  passed by reference
         +---------------+---------------+
   add_guest(e,s)   check_in(e,s)   check_out(e,s)   ... 10 functions
        ^               ^               ^
   AddGuestEvent   CheckInEvent    CheckOutEvent

Files: models.h, storage.h (HotelStorage), faas_functions.h (events + decls),
       faas_functions.cpp (10 bodies), main.cpp, feature_extension.cpp
```

How the two architectures differ

Property	Traditional	FaaS
Data ownership	`HotelSystem` owns it (private)	External `HotelStorage` passed in
Lookup	`map::find()` — O(log n)	linear vector scan — O(n)
State between calls	persists in the object	none; storage is always external
Adding a feature	modify the class	add a function; existing code untouched

Both are stateless *between program runs*; the FaaS version is additionally stateless *between function calls*, which is the property a real cloud platform relies on to scale each function independently.

Part 3 — Feature Extension: Emergency Maintenance + Auto Room Reassignment

New feature: when an emergency maintenance issue is reported, the affected room is taken offline, any active reservation is moved to a free room of the same type, and the guest is notified.

```
report emergency -> room OFFLINE -> find affected reservation
      -> find replacement room (same type) -> reassign -> notify guest
```

Traditional — must modify existing files. A new method `emergencyMaintenance()` was added to `hotel_system.h` *and* `hotel_system.cpp`. One method now touches **three private members** (`rooms_`, `reservations_`, `guests_`) and mixes five responsibilities (report, search reservations, search rooms, reassign, notify). This increases coupling — the class grows with every cross-cutting feature and the change risks breaking existing behaviour because it edits shared code.

FaaS — only new files/functions added. The same feature was implemented in `feature_extension.cpp` as four new independent functions (`find_affected_reservations`, `find_replacement_room`, `reassign_reservation`, `notify_guest`) plus an orchestrator that chains them. **Zero existing functions were modified**, so there is no regression risk and each new function is independently testable.

	Traditional	FaaS
Files changed	2 existing files edited	0 existing; 1 new file
Responsibilities added to old code	5 (in one method)	0
Regression risk	Medium (shared class edited)	Low (isolated additions)
Debugging the full flow	Easy (one call stack)	Harder (multi-step, no atomicity)

Verdict: FaaS is clearly easier to *extend*; Traditional is easier to *debug and keep consistent* because the whole flow is one atomic method.

Part 4 — Performance Evaluation

Workload (identical for both, `benchmark` mode, ~66,000 operations):

10,000 guests, 1,000 rooms, 10,000 reservations, 10,000 check-ins, 10,000 payments, 10,000 check-outs, 10,000 cleaning tasks, 5,000 maintenance reports.

Measured on the course KVM (Ubuntu, `perf stat -r 5`, `/usr/bin/time`):

Metric	Traditional	FaaS	Result
Execution time	**0.0211 s**	0.5971 s	Traditional 28x faster
CPU cycles	**45.5 M**	1,419 M	31x fewer
Instructions	**64.4 M**	1,627 M	25x fewer
Cache-misses	92	93	equal
Context-switches	**3**	13	~4x fewer
Max memory (RSS)	8,268 KB	**5,460 KB**	FaaS 34% smaller

Which performed better and why. The **Traditional** architecture is far faster. The cause is algorithmic, not cosmetic: Traditional indexes data with `std::map` (O(log n)), while the FaaS simulation mimics *external storage access* with `std::vector` linear scans (O(n)). By the last benchmark round the reservation vector holds 10,000 entries, so each `check_in` / `check_out` scans thousands of elements — this matches the 25x extra instructions and 31x extra cycles.

- **Cache-misses are equal** because both datasets fit in L2/L3 cache (<=10 MB); the bottleneck is instruction count, not memory latency.
- **FaaS uses less memory** because `std::vector` stores elements contiguously, whereas each `std::map` node carries red-black-tree pointer overhead (~24 B/entry x six maps).
- **More context-switches for FaaS** simply reflect its longer runtime giving the scheduler more chances to preempt; Traditional finishes in 21 ms.

Interpretation. This does *not* mean FaaS is a worse architecture — the simulation faithfully charges FaaS for the cost of going to external storage. In a real deployment the cloud database provides indexed O(1) lookups, and FaaS wins on independent horizontal scaling once a single machine can no longer hold the monolith''s state.

Conclusion

The two implementations are functionally identical but structurally opposite. Traditional wins on raw single-machine performance (28x faster, driven by in-process indexed lookups) and on consistency. FaaS wins on extensibility (the Part 3 feature needed zero edits to existing code) and on memory footprint. The measurements confirm the standard architectural trade-off: monoliths are faster and simpler in-process, while FaaS trades

per-call overhead for isolation, independent scaling, and low-risk evolution.

Appendix — AI Tool Usage

An AI assistant (GitHub Copilot) was used only for **architectural discussion and code organization**: brainstorming the Part-3 room-reassignment feature, structuring the code into the multi-file layout, and helping interpret the `perf` output. The scenario choice, benchmark design, final code, and all written analysis were decided and verified by the student. No external repository implementation was copied.